Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №4

По дисциплине: «Модели решения задач в интеллектуальных системах»

Тема: «Классификация линейно разделимых образов однослойным персептроном для логических функций n переменных»

Выполнил:

Студент 3 курса

Группы ИИ-26(1)

Пасевич К.Ю.

Проверила:

Андренко К.В.

Брест 2026

Цель работы: реализовать нейронную сеть (однослойный персептрон) для решения задач синтеза логических функций от n входов. Сравнить эффективность обучения при различных начальных условиях и параметрах шага.

Ход работы

1. Сгенерировать полную таблицу истинности для заданной логической функции (2n наборов). На выходе функции должны использоваться значения 0 и 1.
2. Реализовать алгоритм обучения:
    - Разделить выборку на обучающую (часть данных) и тестовую (оставшиеся наборы) для проверки обобщающей способности.
    - В качестве функции активации использовать сигмоиду
    - Провести два эксперимента по обучению:

А. С фиксированным шагом обучения

Б. С адаптивным шагом

- Выход сети интерпретировать как вероятность принадлежности к классу «1».
- Обучение проводить до минимизации суммарной ошибки на обучающей выборке.

3. Провести анализ сходимости:
    - Построить графики зависимости суммарной ошибки от номера эпохи для обучающей и тестовой выборок.
    - Оценить обобщающую способность: корректно ли сеть предсказывает значения на тех наборах из таблицы истинности, которые она не видела при обучении.
    - Зафиксировать количество эпох, необходимых для настройки весов при разных значениях
4. Вывести итоговые значения весовых коэффициентов и порога.
5. Реализовать режим функционирования:
    - Программа должна принимать на вход вектор из n чисел (0 или 1).
    - Сеть должна мгновенно выдавать результат (вероятность и округленный класс).
6. Сделать вывод о способности сигмоидального персептрона представлять базовые логические операции и восстанавливать полную таблицу истинности по частичной выборке.

Допускается применение математических и графических библиотек (NumPy, Matplotlib). Использование ML-фреймворков (PyTorch, Keras и т.д.) запрещено.

Вариант 9

№ of variant	n	Task
9	7	OR

Код программы:

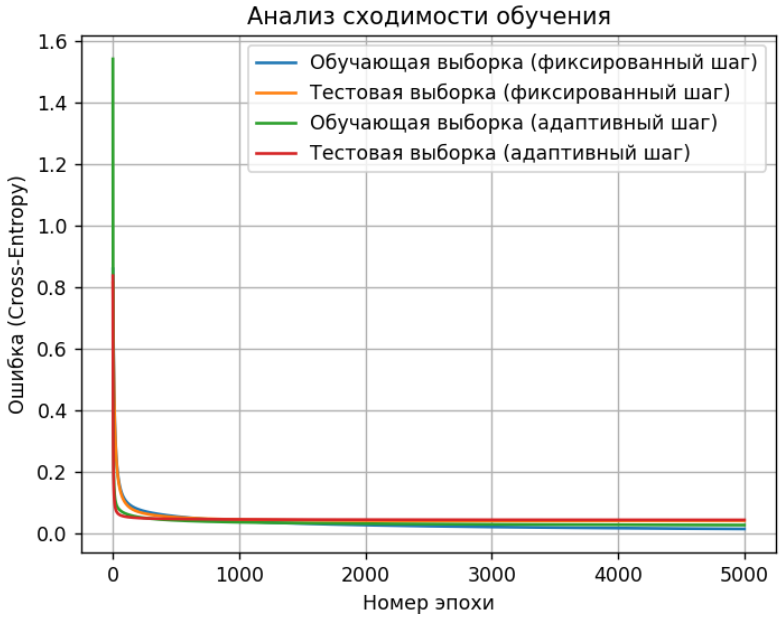
```
import numpy as np
import matplotlib.pyplot as plt
from itertools import product

# 1. ПАРАМЕТРЫ
n = 7
test_ratio = 0.3
max_epochs = 5000
patience = 100
min_delta = 1e-5

# 2. ДАННЫЕ (OR)
X = np.array(list(product([0, 1], repeat=n)))
y = np.array([int(any(x)) for x in X])

print(f"Сгенерирована таблица истинности: {len(X)} наборов")

# 3. РАЗДЕЛЕНИЕ ВЫБОРКИ
np.random.seed(42)
indices = np.random.permutation(len(X))

split = int(len(X) * (1 - test_ratio))
train_idx = indices[:split]
test_idx = indices[split:]

X_train, y_train = X[train_idx], y[train_idx]
X_test, y_test = X[test_idx], y[test_idx]

print(f"Обучающая выборка: {len(X_train)}")
print(f"Тестовая выборка: {len(X_test)}")

# 4. СИГМОИДА
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# 5. ФУНКЦИЯ ПОТЕРЬ
def loss(y_true, y_pred):
    eps = 1e-9
    return -np.mean(
        y_true * np.log(y_pred + eps) +
        (1 - y_true) * np.log(1 - y_pred + eps)
    )

# 6. ОБУЧЕНИЕ
def train_model(X_train, y_train, X_test, y_test, lr=0.1, adaptive=False):
    np.random.seed()

    w = np.random.randn(n)
```

```
    b = -1.0

    train_errors = []
    test_errors = []

    best_loss = float('inf')
    patience_counter = 0

    for epoch in range(max_epochs):
        z = X_train @ w + b
        y_pred = sigmoid(z)

        # балансировка классов
        class_weights = np.where(y_train == 0, 20, 1)

        grad = (y_pred - y_train) * class_weights

        dw = X_train.T @ grad / len(X_train)
        db = np.mean(grad)

        # шаг обучения
        if adaptive:
            lr_current = lr / (1 + 0.01 * epoch)
        else:
            lr_current = lr

        w -= lr_current * dw
        b -= lr_current * db

        train_loss = loss(y_train, y_pred)
        test_pred = sigmoid(X_test @ w + b)
        test_loss = loss(y_test, test_pred)

        train_errors.append(train_loss)
        test_errors.append(test_loss)

        # early stopping
        if best_loss - train_loss > min_delta:
            best_loss = train_loss
            patience_counter = 0
        else:
            patience_counter += 1

        if patience_counter >= patience:
            print(f"Остановка обучения на эпохе {epoch} (достигнута сходимость)")
            break

    print(f"Итоговая ошибка (обучение): {train_loss:.6f}")
    print(f"Итоговая ошибка (тест): {test_loss:.6f}")

    return w, b, train_errors, test_errors, epoch

# 7. ЭКСПЕРИМЕНТЫ
print("\n=== ЭКСПЕРИМЕНТ 1: ФИКСИРОВАННЫЙ ШАГ ОБУЧЕНИЯ ===")
w_fixed, b_fixed, train_f, test_f, ep_f = train_model(
    X_train, y_train, X_test, y_test, lr=0.1, adaptive=False
)

print("\n=== ЭКСПЕРИМЕНТ 2: АДАПТИВНЫЙ ШАГ ОБУЧЕНИЯ ===")
w_adapt, b_adapt, train_a, test_a, ep_a = train_model(
    X_train, y_train, X_test, y_test, lr=0.5, adaptive=True
```

```
)

# 8. ГРАФИКИ
plt.figure()
plt.plot(train_f, label="Обучающая выборка (фиксированный шаг)")
plt.plot(test_f, label="Тестовая выборка (фиксированный шаг)")
plt.plot(train_a, label="Обучающая выборка (адаптивный шаг)")
plt.plot(test_a, label="Тестовая выборка (адаптивный шаг)")

plt.xlabel("Номер эпохи")
plt.ylabel("Ошибка (Cross-Entropy)")
plt.title("Анализ сходимости обучения")
plt.legend()
plt.grid()
plt.show()

# 9. ОЦЕНКА
def evaluate(X, y, w, b):
    probs = sigmoid(X @ w + b)
    preds = (probs > 0.5).astype(int)
    return np.mean(preds == y)

acc_fixed = evaluate(X_test, y_test, w_fixed, b_fixed)
acc_adapt = evaluate(X_test, y_test, w_adapt, b_adapt)

print("\n=== ОЦЕНКА ОБОБЩАЮЩЕЙ СПОСОБНОСТИ ===")
print(f"Точность (фиксированный шаг): {acc_fixed:.4f}")
print(f"Точность (адаптивный шаг): {acc_adapt:.4f}")

# 10. ВЕСА
print("\n=== ИТОГОВЫЕ ПАРАМЕТРЫ МОДЕЛИ ===")
print("\nФиксированный шаг:")
print("Веса:", w_fixed)
print("Порог (bias):", b_fixed)
print("Число эпох:", ep_f)

print("\nАдаптивный шаг:")
print("Веса:", w_adapt)
print("Порог (bias):", b_adapt)
print("Число эпох:", ep_a)

# 11. РЕЖИМ РАБОТЫ
def predict(x, w, b):
    x = np.array(x)
    prob = sigmoid(np.dot(x, w) + b)
    return prob, int(prob > 0.5)

print("\n=== РЕЖИМ ФУНКЦИОНИРОВАНИЯ СЕТИ ===")
while True:
    user_input = input(f"Введите {n} значений (0/1) или 'exit': ")

    if user_input.lower() == "exit":
        break

    try:
        x = list(map(int, user_input.split()))
    except:
        print("Ошибка ввода")
        continue

    if len(x) != n or any(i not in [0,1] for i in x):
```

```
        print("Ошибка: нужно ввести ровно 7 значений 0 или 1")
        continue

    prob, cls = predict(x, w_fixed, b_fixed)

    print(f"Вероятность принадлежности к классу 1: {prob:.4f}")
    print(f"Результат классификации: {cls}")
```

Результат программы:



Сгенерирована таблица истинности: 128 наборов
Обучающая выборка: 89
Тестовая выборка: 39

=== ЭКСПЕРИМЕНТ 1: ФИКСИРОВАННЫЙ ШАГ ОБУЧЕНИЯ ===
Итоговая ошибка (обучение): 0.014543
Итоговая ошибка (тест): 0.041674

=== ЭКСПЕРИМЕНТ 2: АДАПТИВНЫЙ ШАГ ОБУЧЕНИЯ ===
Итоговая ошибка (обучение): 0.027239
Итоговая ошибка (тест): 0.043923

=== ОЦЕНКА ОБОБЩАЮЩЕЙ СПОСОБНОСТИ ===
Точность (фиксированный шаг): 1.0000
Точность (адаптивный шаг): 1.0000

=== ИТОГОВЫЕ ПАРАМЕТРЫ МОДЕЛИ ===

Фиксированный шаг:
Веса: [4.13689462 2.78606285 3.84431725 4.18220188 3.89020258 3.2977859
 3.12825639]
Порог (bias): -2.7042756599372475

Число эпох: 4999

Адаптивный шаг:
Веса: [2.92141326 2.24963871 2.71474213 2.88381686 2.80841147 2.38563393
 2.40067502]
Порог (bias): -1.9998178609997093
Число эпох: 4999

=== РЕЖИМ ФУНКЦИОНИРОВАНИЯ СЕТИ ===
Введите 7 значений (0/1) или 'exit': 0 0 0 0 0 0 0
Вероятность принадлежности к классу 1: 0.0627
Результат классификации: 0
Введите 7 значений (0/1) или 'exit': 1 1 1 1 1 1 1
Вероятность принадлежности к классу 1: 1.0000
Результат классификации: 1

Вывод: реализовала нейронную сеть (однослойный персептрон) для решения задач синтеза логических функций от n входов. Сравнила эффективность обучения при различных начальных условиях и параметрах шага.